

Mapping Ghaf Tree Crowns from UAV Imagery

A step-by-step operating handbook

Prosopis cineraria (Ghaf) crown delineation from area-wide UAV orthomosaics, using the trained FastViT-MA36 + Mask2Former model.

Written for a reader who is comfortable following instructions and does not write code. Every step is a command to copy, paste and run, with what it does, how long it takes, and what a correct result looks like.

Repository: github.com/brakuta/Ghaf-Tree-Crown-Mapping-from-UAV-Data

Accompanies the manuscript "Hybrid Vision-CNN Architecture for Mapping *Prosopis cineraria* from Area-wide UAV-based Images"

First issue, September 2026

Contents

Contents	2
1. How to use this handbook	4
Conventions	4
If you are in a hurry	4
2. What this system does	5
What it is good at, and what it is not	5
The three outputs, in plain terms	5
3. What you were given	6
The folder you received	6
What the computer needs	6
4. Words you will meet	8
5. Installing, once	9
Step 0 - open the right window	9
Step 1 - create the environment and enter it	9
Step 2 - install PyTorch	9
Step 3 - install the OpenMMLab framework	9
Step 4 - install this project	10
A note about paths with spaces or "&"	10
6. Proving that it works	11
Check 1 - the code	11
Check 2 - the models	11
Check 3 - the data	12
7. Mapping crowns in one image	13
The command	13
Is the answer sensible?	13
Adjustments worth knowing	14
The full survey mosaic	14
Cutting your own sample	14
8. Mapping a whole folder of images	15
What it writes	15
Switches for a long batch	15
9. Looking at the results	17
Opening the crowns in QGIS	17

Opening the rasters	17
Numbers to sanity-check against	17
10. Scoring a model against the labelled tiles	18
Per-tile predictions over a labelled split	18
11. Training and fine-tuning	19
Training from scratch	19
Fine-tuning on labels from a new site	19
12. When something goes wrong	20
The method: five steps	20
Before you search: three questions	20
Errors you may actually meet	21
How to search for an answer	23
Asking a person for help	24
13. Quick reference	25
Checks	25
Mapping crowns	25
Scoring, predicting a split, training	25
Getting out of trouble	25
Where to read more	25

1. How to use this handbook

This handbook takes you from a computer with nothing installed to crown polygons you can open in QGIS. It assumes no programming. It does assume you will read the screen: almost everything that goes wrong announces itself in the last two or three lines of output, and chapter 12 is a method for turning those lines into a fix.

Conventions

A shaded box is something you type (or paste) and run. Press Enter after it, and wait for the prompt to come back before typing the next one:

```
python tools\smoke_test.py
```

A box with a green edge is what a correct run prints. Yours will not match character for character - times and file paths differ - but the shape and the key numbers should:

Example

```
all 6 model(s) built and ran a forward pass
```

Paste one command at a time. Some terminals join a multi-line paste into a single line and run something you did not intend. If a command in this handbook wraps onto a second line, it is still **one** command: the wrap is the page being narrow, not a second line to type. Join the pieces with a single space.

Paths in this handbook are written as D:\ghaf-project\... Substitute wherever your copy actually lives. Nothing else changes. Commands are written for the Windows Command Prompt; on macOS or Linux they are identical except that paths use / instead of \.

If you are in a hurry

Chapters 5 and 6 install and prove the installation; chapter 7 maps an image. Those three are the shortest path to a result. Everything else can wait until you need it.

2. What this system does

You give it an aerial image. It gives you back the Ghaf tree crowns in that image, as shapes on a map.

Underneath, a neural network looks at the image in 1024 x 1024 pixel windows and decides, for every single pixel, whether that pixel is part of a Ghaf crown or not. The windows overlap by half their width and the answers are blended where they overlap, so there are no visible seams between windows. The result is then traced into polygons - one polygon per crown - each carrying its area in square metres.

The model in use is **FastViT-MA36 + Mask2Former**, which scored **79.32 mIoU** and **87.22 F1** on the held-out test tiles. Five other models are provided for comparison; unless you have a reason, use the FastViT one.

What it is good at, and what it is not

It can	It cannot
Map crowns in an orthomosaic of any size, from a small plot to a billion-pixel survey	Tell one tree species from another beyond Ghaf and "not Ghaf" - it was trained on two classes only
Return crowns as GIS polygons with areas, ready to count and measure	Work on imagery that is not red-green-blue 8-bit; other band arrangements need - -bands
Reproduce the published scores exactly, from the weights you were given	Separate two crowns that physically touch and overlap into two polygons - they may merge into one
Be re-trained or fine-tuned on labelled tiles from a new site	Run usefully fast without an NVIDIA GPU, though it will run on a CPU

The three outputs, in plain terms

Output	What it is	What it is for
<code>crowns.gpkg</code>	A GeoPackage: the crowns as polygons, with an <code>area_m2</code> column	Counting trees, measuring crowns, joining to other GIS data. This is the one most people want
<code>crowns.tif</code>	A picture the same size as the input where each pixel is 1 (crown) or 0 (not)	Overlaying on the imagery, computing canopy cover, differencing against a labelled mask
<code>probability.tif</code>	The same shape, but each pixel is the model's confidence from 0.00 to 1.00	Choosing a stricter or looser cut-off after the fact, and seeing where the model was unsure

3. What you were given

Three parts, kept separate on purpose. The code is public; the weights and the imagery are not.

Part	What it is	Where it comes from
The code	Configurations, tools, documentation	Public on GitHub
The models	Six trained checkpoints, one folder each	Shared with you directly. Not on GitHub
The data	Labelled tiles, and sample imagery to run on	Shared with you directly. Not on GitHub

The folder you received

The layout assumed throughout this handbook

```

D:\ghaf-project\
+-- README.md           what this is; start here
+-- MANIFEST.json       every part, its size, and what was checked
+-- code\              the repository: tools, configs, docs
+-- models\           2.2 GB
|   +-- fastvit-ma36_mask2former\
|   |   +-- fastvit-ma36_mask2former.py   the complete recipe, self-contained
|   |   +-- best_mIoU_iter_3500.pth       the trained weights
|   |   +-- metadata.json                fingerprint, size, scores
|   |   +-- ...                          five more models
+-- init-weights\      0.9 GB ImageNet weights, for training offline
+-- data\ghaf\        15.6 GB
|   +-- training\{images,masks}\         7005 tile pairs
|   +-- validation\{images,masks}\       869 tile pairs
|   +-- testing\ghaf26\{images,masks}\   767 tile pairs
+-- samples\
|   +-- Kalba26_sample.tif               a small clip: start here, minutes
+-- predictions\testing\                predictions already made for the test split

```

Tiles are 1024 x 1024 PNG pairs: an image and a mask sharing a filename. In a mask, 0 is background and 1 is a Ghaf crown.

The test tiles have small companion files beside them (.pgw, .png.aux.xml, .ovr). These hold the patch of ground each tile covers, and display pyramids, so predictions made from them open in QGIS already in the right place. The training and validation tiles have none, which changes nothing about training or scoring. Do not delete or move these companion files.

MANIFEST.json is the receipt. It records every part of the bundle, its size, the number of files, and the exact version of the code it was built from. If you ever need to prove that a copy is complete, or work out which version of the code produced a result, it is in there.

What the computer needs

Item	Needed	Notes
Operating system	Windows 10/11, macOS or Linux	This handbook shows Windows commands

Item	Needed	Notes
GPU	An NVIDIA GPU, 8 GB or more	The published work used an RTX A5000. Without a GPU everything still runs with <code>--device cpu</code> , perhaps 20-50 times slower
Disk for the bundle	About 20 GB	Code, models, initialisation weights, tiles, samples
Disk for a run	9 bytes per pixel of the image being mapped	An 8192 x 8192 clip needs 0.6 GB; the full 84 072 x 103 691 mosaic needs about 79 GB. Freed when the run ends
Software	Miniconda or Anaconda	Free, from docs.conda.io. Everything else is installed by the commands in chapter 5

4. Words you will meet

Skim this once. You do not need to memorise it; come back when a word in a later chapter is unfamiliar.

Word	What it means here
Orthomosaic	One large image made by stitching together the hundreds of photographs taken on a drone flight, geometrically corrected so that it can be measured like a map
Georeferenced	The file knows where on Earth it sits. Two layers that are both georeferenced line up automatically in QGIS
CRS	Coordinate Reference System - the map projection the coordinates are in. This project's imagery is EPSG:32640 (UTM zone 40 North), which covers the UAE
GSD	Ground sampling distance: how much ground one pixel covers. The sample mosaic is about 2.7 cm per pixel
Tile	A small square cut out of a large image. Training uses 1024 x 1024 tiles
Mask	An image where the pixel value is a label rather than a colour. Here, 0 = background, 1 = Ghaf
Checkpoint (.pth)	A file holding a trained model's weights - what it learned. Roughly 300-800 MB each
Config (.py)	The complete recipe for a model: its architecture, the data pipeline, the training schedule
Inference	Using a trained model to make predictions. The opposite of training
mIoU / F1	Accuracy scores between 0 and 100. Higher is better. They measure how well predicted crowns overlap the hand-drawn ones
Threshold	The confidence above which a pixel is called a crown. The default is 0.50
Environment (conda)	A private, self-contained installation of Python and its packages, so this project cannot break anything else on the computer. Ours is named ghaf
Terminal / Command Prompt	The window where you type commands. On Windows, use the Anaconda Prompt
GeoPackage (.gpkg)	A standard GIS file holding shapes and their attributes. Opens in QGIS and ArcGIS
Scratch space	Temporary disk the program uses while working, released when it finishes

5. Installing, once

This takes about twenty minutes on a good connection and is done once per computer. Every command goes in the same window.

Step 0 - open the right window

1. Press the Windows key, type **Anaconda Prompt**, and open it.
2. A black or blue window appears with a line ending in **>**. That line is the **prompt**; you type after it.
3. If the prompt starts with (base), conda is installed and working. If nothing called Anaconda Prompt exists, install Miniconda first from docs.conda.io, then open a new one.

Step 1 - create the environment and enter it

```
conda create -n ghaf python=3.9 -y
```

```
conda activate ghaf
```

The prompt now begins with (ghaf). That is how you know you are in the right place. Every command in this handbook assumes it. If you close the window and open a new one, run `conda activate ghaf` again - this is the single most common cause of confusing errors later.

Step 2 - install PyTorch

This is the exact version the published models were trained with. It is a large download, around 2 GB.

```
python -m pip install torch==1.12.1+cu113 torchvision==0.13.1+cu113 --extra-index-url  
https://download.pytorch.org/whl/cu113
```

No NVIDIA GPU? Use this instead. Everything still runs, just slowly:

```
python -m pip install torch==1.12.1 torchvision==0.13.1
```

Step 3 - install the OpenMMLab framework

Three commands, in this order. The version numbers matter: they are the ones the models were built and tested against.

```
python -m pip install -U openmim
```

```
python -m mim install mmengine==0.10.7 "mmcv>=2.0.0rc4,<2.2.0"
```

```
python -m pip install mmdet==3.3.0 mmpretrain==1.2.0
```

mmcv is the slowest of these - it is fetching a large pre-built package matched to your PyTorch and CUDA versions. Several minutes is normal.

Step 4 - install this project

Move into the code folder first. The /d is needed when changing to another drive letter:

```
cd /d D:\ghaf-project\code
```

```
python -m pip install -r requirements.txt
```

```
python -m pip install -e "[test]"
```

Two messages during installation that are not errors. pip may say that opencv-python requires numpy>=2; ignore it, OpenCV works with either. It may also print warnings about dependency conflicts among packages you are not using. Only a line beginning ERROR: that stops the install matters.

A note about paths with spaces or "&"

If your project folder contains spaces, put the whole path in double quotes. If it contains an ampersand (&), quotes are **required**: without them the Command Prompt treats & as "end of command, start another one", and you get a confusing "The system cannot find the path specified" twice over.

```
cd /d "Z:\Survey Data\Cineraria_Data & Model\ghaf-project\code"
```

6. Proving that it works

Three checks, about five minutes in total. Run them before anything else, and again on any day when something behaves strangely. They confirm that the code, the weights and the tiles all arrived intact and agree with each other.

Check 1 - the code

No GPU and no data needed, about a minute.

```
python -m pytest tests\ -q
```

What you should see

```
.....
.....
325 passed, 1 skipped in 74.19s
```

Any number of passing tests with no F characters and no failed in the last line is a pass. If you see no module named 'mmengine', you are running a different Python than the one you installed into - see chapter 12, it is the first entry.

Check 2 - the models

This builds all six models from their configurations, loads the weights you were given, checks every file against its published fingerprint, and runs a prediction through each. About 90 seconds on a GPU.

```
python tools\smoke_test.py --checkpoints D:\ghaf-project\models
```

What you should see

```
model                                digest                                weights  prediction
-----
fastvit-ma36_mask2former             ok                                all 1,558 matched  0.00% ghaf
poolformer-s36_fpn                   ok                                all 436 matched   0.00% ghaf
dpn98_fpn                             ok                                all 676 matched   0.00% ghaf
convnext-small_upernet               ok                                all 430 matched   0.00% ghaf
resnet-50_mask2former                ok                                all 610 matched   0.00% ghaf
efficientnet-b3_fpn                  ok                                all 610 matched   0.00% ghaf

all 6 model(s) built and ran a forward pass
```

Column	What it proves
digest = ok	The checkpoint file is byte-for-byte the one that was released. Nothing was corrupted in the copy
all N matched	Every weight in the file found its place in the model built from the configuration - nothing missing, nothing left over
+0 in the delta column	The number of parameters matches the published figure exactly
0.00% ghaf	Correct. This step predicts on a blank synthetic tile, not on imagery, so finding no trees is what should happen. Real imagery gives a few percent

Every command that loads a model prints screens of UserWarning lines - about `__floordiv__`, `torch.meshgrid`, `binary segmentation` and `build_loss`. They come from inside PyTorch and `mmsegmentation`, they appear on a perfectly correct run, and there is nothing to do about them. Read past them to the table at the end.

Check 3 - the data

```
python tools\check_dataset.py D:\ghaf-project\data\ghaf
```

What you should see

```
split      images    masks    paired    checked    status
-----
training    7005      7005      7005      200      ok
validation  869       869       869       200      ok
testing     767       767       767       200      ok

8,641 paired tile(s) across 3 split(s)
dataset looks usable
```

Variation	When to use it
<code>check_dataset.py PATH --full</code>	Opens every one of the 8641 tiles instead of a sample of 200. Slow, thorough; worth doing once
<code>check_dataset.py PATH --sample 0</code>	Checks only that every image has a matching mask, without opening any. Seconds, even over a network drive
<code>check_dataset.py PATH --sample 25</code>	Opens 25 tiles per split. A good compromise after copying the data somewhere new

7. Mapping crowns in one image

This is the main thing the models are for: give it a UAV orthomosaic, get back a canopy map. The image can be far larger than the computer's memory - it is read in overlapping windows and blended, so nothing has to fit at once.

Start with the small clip. `Kalba26_sample.tif` is a piece cut from the full survey. It runs in a couple of minutes and produces exactly the same kind of output, so it proves the whole chain works before you commit to a run of hours. Only once that has worked is the full mosaic worth starting.

The command

Move into the code folder first, so the rest of the command stays short:

```
cd /d D:\ghaf-project\code
```

```
python -m ghaf.inference.large_image
    ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
    ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth ..\samples\Kalba26_sample.tif
    --out-mask ..\output\crowns.tif --out-prob ..\output\probability.tif --out-polygons
    ..\output\crowns.gpkg --batch-size 4
```

Reading it left to right: the **recipe**, the **weights**, the **image**, then where to put each of the three outputs. `--batch-size 4` processes four windows at a time, which is faster on a GPU with memory to spare.

What you should see

```
INFO Kalba26_sample.tif: 8192 x 8192 px, 225 window(s) of 1024 px (overlap 512), batch 4, 0.6 GB
    scratch
Kalba26_sample: 100%|#####| 57/57 [00:40<00:00, 1.42batch/s]
INFO Created 27 records
INFO wrote ..\output\crowns.gpkg (27 polygon(s))
INFO canopy: 650055 of 67108864 valid px (0.97%)
```

Between 40 seconds and two and a half minutes on one GPU, depending on how many outputs are written and whether they go to a local disk or a network drive. The progress bar is the honest guide to how much is left.

Is the answer sensible?

Number	Expected	If it is far off
Canopy percentage	About 1% on this clip. Scattered Ghaf in desert is a low number by nature	0.00% means the run found nothing; above 50% means it found everything. Both point upstream: usually the wrong checkpoint, or bands that are not red, green, blue
Polygon count	27 on this clip at the default settings	A wildly larger number usually means single-pixel specks - see <code>--min-area</code> below
Crown areas	Roughly 2 to 112 m2 for real crowns	Hundreds of square metres suggests crowns merged, or the image is not at the resolution the model expects

Adjustments worth knowing

Option	Effect
--threshold 0.6	Stricter: fewer, more confident crowns. 0.4 is more inclusive. Default 0.5
--min-area 1	Drop crown polygons under 1 m2. Removes the stray single-pixel specks that any threshold leaves behind, which otherwise inflate a crown count. On the sample clip this removes 11 of 27 "crowns" and leaves the 16 real ones
--batch-size 4	Faster on a GPU with spare memory. Lower it to 1 if you see CUDA out of memory
--device cpu	Run without a GPU. Much slower, but it works
--scratch-dir E:\scratch	Put the temporary working files on a bigger or faster drive
--bands 1 2 3	Which bands of the image are red, green and blue. Only needed for unusual imagery

The full survey mosaic

Kalba26.tif is 84 072 x 103 691 pixels - 8.7 billion of them. At 9 bytes per pixel that is about **79 GB of temporary space**, and roughly 33 500 windows to predict: a run of hours rather than minutes on one GPU. Point --scratch-dir at a drive with room, start it, and leave it. The program checks the free space before it begins rather than failing part-way through.

Cutting your own sample

To try a different part of a mosaic, or to make a quick sample from a new survey, cut one out first. A minute or two:

```
python tools\make_sample.py ..\samples\Kalba26.tif --output ..\samples\my_sample.tif --size 8192
--origin 30000 40000
```

--size is the clip in pixels. --origin is its top-left corner; leave it out and the clip is taken from the centre. The tool reports how much of the clip is real imagery rather than the transparent border around a survey, so a badly placed window is obvious immediately.

8. Mapping a whole folder of images

When you have many images rather than one - a season's clips, a set of survey plots, the frames from one flight - point the batch tool at the folder and leave it. Each image is windowed exactly as a full mosaic is, so they need not be the same size and none of them has to fit in memory.

```
cd /d D:\ghaf-project\code
```

```
python tools\predict_folder.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
    ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth D:\my-images --out-dir
    ..\output\folder --batch-size 4 --min-area 1
```

What you should see

```
INFO 1 image(s) in ..\samples -> ..\output\folder
INFO [1/1] Kalba26_sample.tif
INFO Kalba26_sample.tif: 8192 x 8192 px, 225 window(s) of 1024 px (overlap 512), batch 4, 0.6 GB
    scratch
INFO dropped 11 polygon(s) under 1 m2
INFO wrote ..\output\folder\polygons\Kalba26_sample.gpkg (16 polygon(s))
INFO canopy: 650055 of 67108864 valid px (0.97%)
INFO 1 predicted, 0 skipped, 0 failed; canopy 0.97% of valid px
INFO wrote ..\output\folder\summary.json
```

What it writes

Output layout

```
folder\
+-- polygons\      one .gpkg of crowns per image, with area_m2      <- the output
+-- masks\         the 0/1 raster, only if you add --save-mask
+-- probability\   the confidence raster, only with --save-probability
+-- summary.json   every image, its canopy share, and any failures
```

You get crowns, not rasters. A GeoPackage of crowns is a few hundred kilobytes where the mask behind it is hundreds of megabytes, and it is the crowns you count, measure and drape over the imagery. The mask is still made - the polygons are traced from it - but it is written to temporary space and deleted when that image is done. Add `--save-mask` when you want to keep it.

If the input folder has subfolders, the output mirrors them, so two images with the same name in different folders cannot overwrite each other.

Switches for a long batch

Switch	Effect
<code>--limit 3</code>	Stop after three images. Always do this first on a big folder: it proves the settings before you commit to hundreds
<code>--recursive</code>	Include images in subfolders
<code>--pattern "*_rgb.tif"</code>	Only files whose name matches. Useful when a folder mixes RGB images with elevation models
<code>--skip-existing</code>	Carry on where an interrupted run stopped, instead of redoing finished images

Switch	Effect
--save-mask	Also keep the 0/1 raster per image
--save-probability	Also keep the confidence raster per image
--min-area 1	Drop crowns under 1 square metre

One bad file does not stop the run. If an image is unreadable or has an unexpected number of bands, that image is reported, the batch carries on, and the failure is listed in `summary.json` at the end. Read `summary.json` after any long run: it names anything that did not work.

9. Looking at the results

Everything the tools write carries the coordinate system of the image it came from, so it lands in the right place on the map with no manual positioning.

Opening the crowns in QGIS

1. Open QGIS. **Layer > Add Layer > Add Vector Layer**, browse to `crowns.gpkg`, and click **Add**.
2. Add the imagery underneath it: **Layer > Add Layer > Add Raster Layer**, and choose the image you mapped. Drag it below the crowns in the Layers panel.
3. Right-click the crowns layer > **Properties > Symbolology**. Set **Fill style** to **No brush** and pick a bright outline colour, so you can see the imagery through each crown.
4. Right-click > **Open Attribute Table** to see one row per crown, with `area_m2`. The row count at the top of that window is your tree count.
5. For totals: **Vector > Analysis Tools > Basic Statistics for Fields**, choose the crowns layer and the `area_m2` field. It gives the count, sum, mean, minimum and maximum crown area.

Opening the rasters

`crowns.tif` and `probability.tif` are added the same way, through **Add Raster Layer**. Two tips that save confusion:

- The mask looks black at first, because its values are 0 and 1 and QGIS stretches 0-255 by default. Right-click > **Properties > Symbolology**, set the render type to **Paletted/Unique values**, and click **Classify** - you then get two colours, one per class.
- For the probability raster, use **Singleband pseudocolor** and a colour ramp from 0 to 1. Anywhere yellow-to-red is where the model was most confident.

Numbers to sanity-check against

Quantity	Value seen in this project
Canopy share, sample clip	About 0.97% of valid pixels
Canopy share, test split	About 3.44% across the 767 test tiles
Crown count, sample clip	27 raw; 16 after <code>--min-area 1</code>
Crown areas, sample clip	About 2.4 to 112 m2
Ground sampling distance	About 2.68 cm per pixel
Total crown area, sample clip	About 469 m2, against 467 m2 computed from the mask pixels - the two agree, as they should

Those figures are the yardstick. A result an order of magnitude away from them is worth investigating before it goes into a report.

10. Scoring a model against the labelled tiles

This reproduces the published numbers from the weights you were given - the strongest single check that everything on your machine is correct. About three minutes on a GPU.

```
cd /d D:\ghaf-project\code
```

```
python tools\test.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
    ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --data-root ..\data\ghaf
```

The last line reports mIoU, mDice and mFscore, with a table above it giving the two classes separately.

Model	mIoU	F1
FastViT-MA36 + Mask2Former	79.32	87.22
PoolFormer-S36 + FPN	78.65	86.72
DPN-98 + FPN	78.19	86.35
ConvNeXt-S + UPerNet	78.02	86.20
ResNet-50 + Mask2Former	77.69	85.98
EfficientNet-B3 + FPN	70.77	80.29

Swap the two paths to score any of the other five. If your number differs from the table by more than a rounding error, the cause is almost always the data root pointing somewhere unexpected, or a checkpoint that is not the one it claims to be - check 2 in chapter 6 settles the second case.

Per-tile predictions over a labelled split

Scoring reduces a whole split to a few numbers. If you want the maps behind those numbers - for figures, or to see *where* the model is wrong rather than by how much - predict the split tile by tile. A few minutes for the test split:

```
python tools\predict_split.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
    ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --data-root ..\data\ghaf
    --split testing --out-dir ..\output\predictions --save-probability
```

One predicted mask per tile, encoded exactly like the ground-truth masks (0 background, 1 ghaf), so a prediction and its label can be subtracted directly to make an error map. `--limit 20` runs a quick partial pass first. Expect a canopy fraction around **3.4%** over the test split.

11. Training and fine-tuning

Only worth doing if you are changing something. The six trained models are already provided, and training the published configuration takes many hours on one GPU.

Training from scratch

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf
```

Checkpoints and logs go to `work_dirs\<config-name>\`. The model is scored on the validation split every 3 500 iterations and the best one is kept as `best_mIoU_iter_*.pth`. If a run is interrupted - a power cut, a closed window - continue it, optimiser state and all:

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf --resume
```

Keep `work_dirs\` when a run finishes. It holds the loss curves, the validation history and the exact configuration the run used. Those logs cannot be reconstructed afterwards, and they are what a reviewer asks for.

Fine-tuning on labels from a new site

Starting from a released checkpoint is much cheaper than training from scratch, and usually better. First prepare the new tiles in the same layout - `training/{images,masks}`, `validation/{images,masks}`, 1024 x 1024 PNG pairs, masks containing only 0 and 1 - and check them:

```
python tools\check_dataset.py D:\ghaf-project\data\new-site --full
```

Then fine-tune, with a shorter schedule and a smaller learning rate:

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\new-site
--load-from ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --cfg-options
train_cfg.max_iters=4000 optim_wrapper.optimizer.lr=1e-5
```

Flag	What it does
<code>--load-from</code>	Takes the weights and starts a fresh schedule. This is what fine-tuning means
<code>--resume</code>	Continues an interrupted run of your own, keeping the optimiser state and iteration count. A different thing entirely
<code>--cfg-options</code> <code>train_cfg.max_iters=4000</code>	A shorter schedule: 4000 iterations rather than the full run
<code>--cfg-options</code> <code>optim_wrapper.optimizer.lr=1e-5</code>	A smaller learning rate, so the model adjusts to the new site without forgetting what it already knows

Score the result exactly as in chapter 10, pointing `--data-root` at the new site and the checkpoint at your new `work_dirs\...\best_mIoU_iter_*.pth`.

12. When something goes wrong

Nothing here is dangerous. No command in this handbook deletes your data, and a failed run can always be started again. The skill worth having is reading what the computer said.

The method: five steps

1. **Read the last three lines first.** Python prints the history of what it was doing (the "traceback") and then, on the very last line, what actually went wrong. That last line is the error. Everything above it is context.
2. **Find the error type.** The last line looks like `SomeError: some message`. The part before the colon is the type - `FileNotFoundError`, `RuntimeError`, `ModuleNotFoundError` - and it tells you the category of the problem.
3. **Check the table on the next page.** Most of what happens in practice is in it, with the fix.
4. **Ask the three questions** in "Before you search" below. They resolve a large share of problems without any searching at all.
5. **Then search**, using the recipe below. Copy the error, not your paths.

Copy the error text before you do anything else. Select it in the terminal window and press Enter (in the Anaconda Prompt this copies), then paste it into Notepad. If you close the window or run another command the text is usually gone, and an error you cannot quote is an error nobody can help with.

Before you search: three questions

Question	How to check	Why it matters
Am I in the ghaf environment?	The prompt should start with <code>(ghaf)</code> . If not: <code>conda activate ghaf</code>	The single most common cause of "module not found". A new terminal window always starts outside it
Am I running the right Python?	<code>python -c "import sys; print(sys.executable)"</code> - the path printed must contain <code>envs\ghaf</code>	A base Anaconda install ahead on the PATH will answer instead, and has none of this project's packages
Am I in the right folder?	<code>cd</code> on its own prints where you are. <code>dir</code> lists what is there	Commands here assume the code folder. A path typed relative to somewhere else will not be found

Errors you may actually meet

What you see	What it means	What to do
No module named mmengine / pytest / ghaf	A different Python is running than the one you installed into	Run <code>conda activate ghaf</code> . Verify with <code>python -c "import sys; print(sys.executable)"</code> . Always start commands with <code>python -m</code>
mmseg ... is not installed in conda environment "x"	Wrong environment, or the stack was never installed on this machine	The message names the interpreter it asked. <code>conda activate ghaf</code> , then repeat the command
No module named ftfy	mmsegmentation imports a tokenizer that needs it, although its own metadata does not say so	<code>python -m pip install ftfy regex</code>
RuntimeError: Numpy is not available	NumPy 2 alongside a PyTorch built for NumPy 1	<code>python -m pip install "numpy<2"</code>
opencv-python ... requires numpy>=2 while installing	A note from pip's resolver, not an error	Ignore it. OpenCV works with either
CUDA out of memory	The GPU ran out of room, usually because the batch size is too high or another program is using the card	Lower <code>--batch-size</code> to 2 or 1. Close other GPU programs. Or add <code>--device cpu</code>
not enough scratch space	The drive holding temporary files is too small for this image	Add <code>--scratch-dir</code> pointing at a drive with room. An image needs 9 bytes per pixel
FileNotFoundError naming a path	A file or folder in the command does not exist as typed	Check for a typo, a missing <code>.\</code> , or a path with spaces that needs double quotes
The system cannot find the path specified, printed twice	The path contains an <code>&</code> , which the Command Prompt read as two separate commands	Put the whole path in double quotes
The process cannot access the file because it is being used by another process	QGIS, ArcGIS or another window has the file open	Close the program holding it. If you are deleting a folder, first <code>cd</code> out of it - a terminal sitting inside a folder holds it open
NO TILES from check_dataset.py	The folders exist but hold no <code>.png</code> or <code>.tif</code> tiles	The row names what it found instead; usually the tiles are one folder deeper, or in another format
SHA-256 mismatch from smoke_test.py	A checkpoint does not match the one that was released	That copy is damaged. Copy it again from the original
0.00% ghaf from smoke_test.py	Nothing. That step predicts on a blank tile	Expected. See chapter 6
0.00% canopy on real imagery	The run genuinely found nothing	Usually the wrong checkpoint, or bands that are not red, green, blue. Check the config and checkpoint paths, then try <code>--bands</code>
Screens of UserWarning about <code>__floordiv__</code> , <code>meshgrid</code> , <code>build_loss</code>	Deprecation notices from inside PyTorch and mmsegmentation	Nothing. They appear on a correct run

What you see	What it means	What to do
KeyboardInterrupt	Someone pressed Ctrl+C, or the terminal was closed	Nothing broke. Run the command again; long runs can be resumed with <code>--skip-existing</code> or <code>--resume</code>
The command did something odd after pasting several lines	The terminal joined them into one line	Paste and run one command at a time

How to search for an answer

If the error is not in the table, it is almost certainly something another person has already hit. Searching well is a skill with a recipe.

What to paste into the search box

Take the **last line** of the error. Remove anything specific to your computer - your name, your drive letters, your file names, any long numbers - because those appear in nobody else's error. Add the name of the library it came from. So this:

What you saw

```
File "D:\ghaf-project\code\ghaf\inference\large_image.py", line 322, in predict_large_image
    with rasterio.open(src_path) as src:
rasterio.errors.RasterioIOError: D:\surveys\flight_07\north_block.tif: No such file or directory
```

becomes this:

What to search for

```
rasterio RasterioIOError No such file or directory
```

Where to search, in order

Where	How	Best for
A search engine	Paste the cleaned error. Add mmsegmentation or rasterio or pytorch - whichever library the error came from	Almost everything. Try this first
GitHub issues for the library	Go to the library's GitHub page, click Issues , and search there. Include closed issues - a closed one usually contains the fix	Errors from inside mmsegmentation, mmdcv, mmdet: github.com/open-mmlab/mmdetection
Stack Overflow	Search the error text. Read the accepted answer and the comments under it	General Python, conda and installation problems
This project's own documents	The docs\ folder beside the code: GETTING_STARTED, AREA_WIDE_INFERENCE, MODEL_ZOO, RELEASE_BUNDLE	Anything about this project specifically rather than the libraries it uses
The tool's own help	Add <code>--help</code> to any command, for example <code>python tools\predict_folder.py --help</code>	Remembering what an option is called, and what it does

Judging what you find

- **Check the versions.** An answer written for mmsegmentation 0.x does not apply to 1.2.2, and one for PyTorch 2.x may not apply to 1.12.1. If the page does not say which version it is about, treat it with suspicion.
- **Prefer answers that explain why.** An answer that only says "run this command" and cannot say what it fixes is as likely to break something as to help.
- **Never blindly upgrade.** Advice of the form "just upgrade mmdcv / torch / numpy" will usually break this installation: the versions in chapter 5 were chosen because they work together and match the trained weights. If you think a version must change, write down what you had first.

- **Change one thing at a time**, and re-run the checks in chapter 6 after each change. Two changes at once and you will not know which one helped.

The escape hatch. If an installation gets into a state you cannot explain, delete the environment and build it again from chapter 5. It costs twenty minutes and touches none of your data: `conda deactivate`, then `conda env remove -n ghaf`, then start at Step 1.

Asking a person for help

When you do need to ask someone, send all six of these. An answer usually comes back in one round instead of five:

1. The **exact command** you ran, copied and pasted, not retyped from memory.
2. The **complete output**, from the command down to the last line. Not a screenshot of part of it, and not just the last line.
3. What you **expected** to happen instead.
4. The result of `python -c "import sys; print(sys.executable)"`.
5. The result of `python -m pip list` (or at least the lines for `torch`, `mmcv`, `mmsegmentation`, `mmdet`, `numpy`).
6. Whether the checks in chapter 6 pass **now** - and whether they ever did on this machine.

13. Quick reference

Every routine command in one place. All of them assume you have run `conda activate ghaf` and `cd /d D:\ghaf-project\code` in that window, and that `MODEL` stands for `..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py` and `WEIGHTS` for `..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth`.

Checks

```
python -m pytest tests\ -q
python tools\smoke_test.py --checkpoints ..\models
python tools\check_dataset.py ..\data\ghaf
```

Mapping crowns

```
python -m ghaf.inference.large_image MODEL WEIGHTS image.tif --out-polygons crowns.gpkg
--out-mask crowns.tif --batch-size 4 --min-area 1

python tools\predict_folder.py MODEL WEIGHTS D:\my-images --out-dir ..\output\folder
--batch-size 4 --min-area 1

python tools\make_sample.py big-mosaic.tif --output sample.tif --size 8192
```

Scoring, predicting a split, training

```
python tools\test.py MODEL WEIGHTS --data-root ..\data\ghaf

python tools\predict_split.py MODEL WEIGHTS --data-root ..\data\ghaf --split testing --out-dir
..\output\predictions

python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf

python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\new-site
--load-from WEIGHTS --cfg-options train_cfg.max_iters=4000 optim_wrapper.optimizer.lr=1e-5
```

Getting out of trouble

```
conda activate ghaf
python -c "import sys; print(sys.executable)"
python tools\predict_folder.py --help
conda env remove -n ghaf
```

Where to read more

Document	Covers
code\README.md	The project, the results, the repository layout
code\docs\GETTING_STARTED.md	The same walkthrough as this handbook, in the code folder
code\docs\MODEL_ZOO.md	All six models, per-class scores, training settings

Document	Covers
code\docs\AREA_WIDE_INFERENCE.md	How a mosaic is tiled and blended, and how to tune it
code\docs\RELEASE_BUNDLE.md	How the models folder is assembled and verified
MANIFEST.json	What is in the bundle, its sizes, and the version of the code that produced it

This handbook describes the repository at github.com/brakuta/Ghaf-Tree-Crown-Mapping-from-UAV-Data. The trained weights, the UAV imagery and the labelled tiles are not distributed with the code; they are available from the corresponding author on reasonable request.